華中科技大學

《数字信号处理》实验报告

第一部分：**Matlab** 平台的使用

姓　　名　罗畅

学　　号　U202113940

专业班级　种子 2101 班

实验平台　Matlab R2023b on MacOS

联系电话　13798581194

2023 年 10 月 30 日

目　　录

1 实验目的 1

2 实验内容 1

3 实验过程与结果分析 1

3.1 初识 Matlab 1

3.1.1 帮助与命令 1

3.1.2 计算器 1

3.1.3 变量赋值操作 2

3.1.4 复数运算的基本操作 3

3.2 再识 Matlab 3

3.2.1 Matlab 数组索引 3

3.2.2 Matlab 脚本文件 6

3.2.3 Matlab 与声音相关的函数命令 9

3.3 用 Matlab 处理正弦信号 10

3.3.1 处理正弦信号 10

3.3.2 复数幅度 11

3.4 复指数简介 12

3.4.1 向量化 12

3.4.2 函数 13

3.5 复指数 13

3.5.1 生成正弦信号的 M-文件 13

3.6 线性调频脉冲 chrip 14

3.6.1 chirp 信号的 Matlab 合成方法 14

3.6.2 chirp 的函数 15

3.6.3 声谱图 15

3.6.4 chirp 的声谱图 16

3.6.5 有趣的 chirp 16

4 实验小结 17

1 实验目的

1. 学习基本的 Matlab 命令和语法，包括“帮助”系统
2. 学习编写你自己的 Matlab 的脚本文件，并像命令一样运行它们
3. 学习一点 Matlab 的高级编程技术，例如向量化

2 实验内容

1. 了解 Matlab 基本功能
2. 了解 Matlab 数组索引、脚本文件、与声音相关函数命令等
3. 用 Matlab 处理正弦信号
4. 用 Matlab 处理复指数相关函数
5. 用 Matlab 生成正弦信号的 M-文件
6. 学习线性调频脉冲 chirp 的使用

3 实验过程与结果分析

3.1 初识 Matlab

3.1.1 帮助与命令

启动 Matlab，在命令行依次输入以下命令：

```
doc
helpwin
helpwin plot
helpwin colon %<--- a VERY IMPORTANT notation
helpwin ops
helpwin zeros
helpwin ones
lookfor filter %<--- keyword search
```

打开的均为帮助窗口，命令 `look for filter` 则在命令行中打印所有可用的滤波器。

3.1.2 计算器

在 Matlab 命令行中依次输入以下命令：

```
pi * pi - 10
sin(pi / 4)
ans^2 %<--- "ans" holds the last result
```

得到：

```
>> pi * pi - 10

ans =

   -0.1304

>> sin(pi / 4)

ans =

    0.7071

>> ans^2

ans =

    0.5000

>>
```

3.1.3 变量赋值操作

在 Matlab 命令行中依次输入以下命令：

```
xx = sin(pi / 5);
cos(pi / 5) %<--- assigned to what?
yy = sqrt(1 - xx * xx)
ans
```

得到：

```
>> xx = sin(pi / 5);
>> cos(pi / 5)

ans =

    0.8090

>> yy = sqrt(1 - xx * xx)

yy =

    0.8090

>> ans

ans =

    0.8090

>> |
```

命令 `cos(pi / 5)` 的值被赋给了 ans。

3.1.4 复数运算的基本操作

在 Matlab 命令行中依次输入以下命令：

```
zz = 3 + 4i, ww = -3 + 4j
real(zz), imag(zz)
abs([zz, ww]) %<-- Vector constructor
conj(zz + ww)
angle(zz)
exp(j * pi)
exp(j * [pi / 4, 0, -pi / 4 ])
```

则会得到每一项的运算结果。

3.2 再识 Matlab

3.2.1 Matlab 数组索引

冒号的含义

在 Matlab 命令行中依次输入以下命令：

```
jkl = 0 : 6
jkl = 2 : 4 : 17
jkl = 99 : -1 : 88
ttt = 2 : (1/9) : 4
tpi = pi * [ 0:0.1:2 ]; %注意这里有分号
```

得到结果如下：

```
>> jkl = 0 : 6
jkl =
     0     1     2     3     4     5     6
>> jkl = 2 : 4 : 17
jkl =
     2     6    10    14
>> jkl = 99 : -1 : 88
jkl =
    99    98    97    96    95    94    93    92    91    90    89    88
>> ttt = 2 : (1/9) : 4
ttt =
  Columns 1 through 10
    2.0000    2.1111    2.2222    2.3333    2.4444    2.5556    2.6667    2.7778    2.8889    3.0000
  Columns 11 through 19
    3.1111    3.2222    3.3333    3.4444    3.5556    3.6667    3.7778    3.8889    4.0000
>> tpi = pi * [ 0:0.1:2 ];
>> tpi
tpi =
  Columns 1 through 10
         0    0.3142    0.6283    0.9425    1.2566    1.5708    1.8850    2.1991    2.5133    2.8274
  Columns 11 through 20
    3.1416    3.4558    3.7699    4.0841    4.3982    4.7124    5.0265    5.3407    5.6549    5.9690
  Column 21
    6.2832
fx >> |
```

可以知道：冒号的作用是“步进”，具体格式是：`start : step : end`，从 start 开始，向 end 步进，每一步的步距是 step，没有定义步距时，默认为 1，最后生成的是一个数组。

从向量提取/插入数字

在 Matlab 命令行中依次输入以下命令：

```
xx = [zeros(1,3), linspace(0,1,5), ones(1,4)]
xx(4 : 6)
size(xx)
length(xx)
xx(2 : 2 : length(xx))
xx(2 : 2 : end)
```

得到结果如下：

```
>> xx = [zeros(1,3), linspace(0,1,5), ones(1,4)]
xx =
  Columns 1 through 10
         0         0         0         0    0.2500    0.5000    0.7500    1.0000    1.0000    1.0000
  Columns 11 through 12
    1.0000    1.0000
>> xx(4 : 6)
ans =
         0    0.2500    0.5000
>> size(xx)
ans =
     1    12
>> length(xx)
ans =
    12
>> xx(2 : 2 : length(xx))
ans =
         0         0    0.5000    1.0000    1.0000    1.0000
>> xx(2 : 2 : end)
ans =
         0         0    0.5000    1.0000    1.0000    1.0000
>> |
```

可以知道：

第一句代码创建了一个名为 xx 的数组，它由三个部分组成：一个由 3 个零组成的数组，一个从 0 到 1 均匀分布的包含 5 个元素的数组，以及一个由 4 个 1 组成的数组。这些部分使用方括号拼接在一起，形成一个整体数组；

第二句代码从数组 xx 中提取索引位置 4 到 6 的元素，生成一个新的数组。在 Matlab 中，数组索引是从 1 开始的，所以这个操作提取的是 xx 中的第 4、第 5 和第 6 个元素；

第三句代码返回数组 xx 的大小，它返回一个包含两个元素的元组，依次表示行数和列数。在这个特定情况下，size(xx) 将返回 (1, 12)，因为 xx 是一个行向量，具有 1 行和 12 列；

第四句代码返回数组 xx 的长度，也就是数组中元素的总数。在这个情况下，length(xx) 将返回 12，因为 xx 中有 12 个元素；

第五句代码使用切片操作，从数组 xx 中提取以 2 为步长的索引位置，从第 2 个元素开始一直到数组的长度。它生成一个新的数组，包含了 xx 中的所有偶数索引位置的元素；

第六句代码与前一个操作类似，它也使用切片操作，提取了以 2 为步长的索引位置，从第 2 个元素开始一直到数组的末尾。这将生成一个新的数组，包含了 xx 中的所有偶数索引位置的元素，但使用了 end 来表示数组的末尾。

替换向量中的值

在 Matlab 命令行中依次输入以下命令：

```
yy = xx;
yy(4:6) = pi*(1:3)
```

得到结果如下：

```
>> yy = xx;
>> yy(4:6) = pi * (1 : 3)
yy =
  Columns 1 through 10
         0         0         0    3.1416    6.2832    9.4248    0.7500    1.0000    1.0000    1.0000
  Columns 11 through 12
    1.0000    1.0000
>>
```

可以知道，第一句代码从数组 xx 复制得到一个新的数组 yy，并将数组 yy 中第 4 到第 6 个元素依次改成了 π 乘以 1-3。

要想完成：

> 现在写一条语句,xx 用 (b) 中定义的方法，把 xx 的偶数索引的元素 (即 xx(2),xx(4) 等) 的值替换为常数 π^{π}。使用向量替换，不要用循环。

可以尝试代码：

```
xx(2 : 2 : end) = pi^pi
```

得到结果如下：

```
>> xx
xx =
  Columns 1 through 10
         0         0         0         0    0.2500    0.5000    0.7500    1.0000    1.0000    1.0000
  Columns 11 through 12
    1.0000    1.0000
>> xx(2 : 2 : end) = pi^pi
xx =
  Columns 1 through 10
         0   36.4622         0   36.4622    0.2500   36.4622    0.7500   36.4622    1.0000   36.4622
  Columns 11 through 12
    1.0000   36.4622
>>
```

可以看到，成功将 xx 的偶数索引的元素的值替换为了常数 π^{π}。

3.2.2 Matlab 脚本文件

关于向量的实验

在 Matlab 命令行中输入以下命令：

```
xk = cos(pi * (0 : 11) / 4)
```

得到结果如下：

```
>> xk = cos(pi * (0 : 11) / 4)
xk =
  Columns 1 through 10
    1.0000    0.7071    0.0000   -0.7071   -1.0000   -0.7071   -0.0000    0.7071    1.0000    0.7071
  Columns 11 through 12
    0.0000   -0.7071
>>
```

其中，`0:11` 生成了 0-11 的整数范围，即 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]；

`pi * (0:11) / 4` 将这些整数值乘以 π，得到一组角度值的弧度表示；

`cos(pi * (0:11) / 4)` 计算了每个角度值的余弦值，并将这些余弦值存储在向量 xk 中。

xk(1) 代表 xk 第一个数，没有 xk(0)。

Matlab 中的循环

在 Matlab 命令行中依次输入以下命令：

```
yy = [ ]; %<--- initialize the yy vector to be empty
for k = -5 : 5
yy(k + 6) = cos(k * pi / 3)
end
yy
```

得到结果如下：

可以看到，向量 yy 的第 1-11 项被赋予了 $cos(k\pi/3)$ 的值。

需要写 yy(k+6) 的原因是向量的下标从 1 开始，如果用 yy(k)，则会报错：

```
>> y = [ ]; %<--- initialize the yy vector to be empty
>> for k = -5 : 5
yy( k + 6) = cos(k * pi / 3)
end
yy =
    0.5000
yy =
    0.5000   -0.5000
yy =
    0.5000   -0.5000   -1.0000
yy =
    0.5000   -0.5000   -1.0000   -0.5000
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000    1.0000
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000    1.0000    0.5000
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000    1.0000    0.5000   -0.5000
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000    1.0000    0.5000   -0.5000   -1.0000
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000    1.0000    0.5000   -0.5000   -1.0000   -0.5000
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000    1.0000    0.5000   -0.5000   -1.0000   -0.5000    0.5000
>> yy
yy =
    0.5000   -0.5000   -1.0000   -0.5000    0.5000    1.0000    0.5000   -0.5000   -1.0000   -0.5000    0.5000
>>
```

```
>> for k = -5 : 5
yy(k) = cos(k * pi / 3)
end
Array indices must be positive integers or logical values.
>>
```

使用冒号完成相同的功能则可以输入以下语句：

```
yy = cos(pi * (-5:5) / 3)
```

得到结果同以上的循环的代码。

基本绘图命令

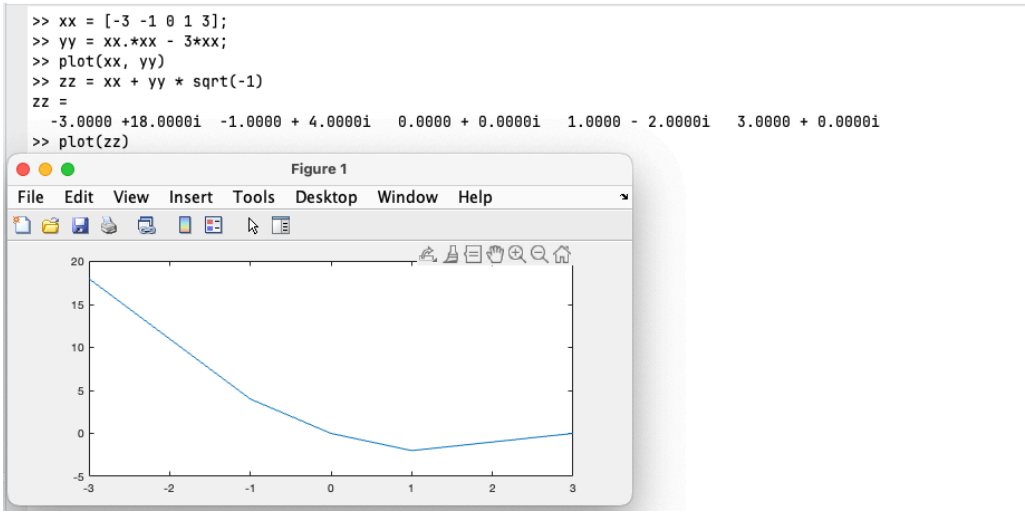
在 Matlab 命令行中依次输入以下命令：

```
xx = [-3 -1 0 1 3];
yy = xx.*xx - 3*xx;
plot(xx, yy)
zz = xx + yy * sqrt(-1)
plot(zz) %<---- complex values: plot imag vs. real
```

得到结果如下：

可以知道：命令 `plot(xx, yy)` 和命令 `plot(zz)` 均绘制了一个以 xx 为横坐标，yy 为纵坐标的图形。

如果 xx 是一个向量，则命令 `xx.*xx` 会让向量的每一个值都变为本身的平方，即 $[x_1, x_2, x_3, ...x_n] \rightarrow [x_1^2, x_2^2, x_3^2, ...x_n^2]$



如果 xx 是一个矩阵，则命令 `xx.*xx` 会让矩阵的每一个值都变为本身的平方，即：

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & \cdots & x_{1n} \\ x_{21} & x_{22} & x_{23} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & x_{m3} & \cdots & x_{mn} \end{bmatrix} \Rightarrow \begin{bmatrix} x_{11}^2 & x_{12}^2 & x_{13}^2 & \cdots & x_{1n}^2 \\ x_{21}^2 & x_{22}^2 & x_{23}^2 & \cdots & x_{2n}^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1}^2 & x_{m2}^2 & x_{m3}^2 & \cdots & x_{mn}^2 \end{bmatrix}$$

Matlab 脚本文件

创建 Matlab 脚本文件，命名为 mylabl.m 输入以下命令：

```
clear all;
close all;
tt = -1 : 0.01 : 1;
xx = cos(5 * pi * tt);
zz = 1.4 * exp(j * pi / 2) * exp(j * 5 * pi * tt);
plot(tt, xx, 'b-', tt, real(zz), 'r--') %<--- plot a sinusoid
grid on
title('TEST PLOT of a SINUSOID')
xlabel('TIME (sec)')
```

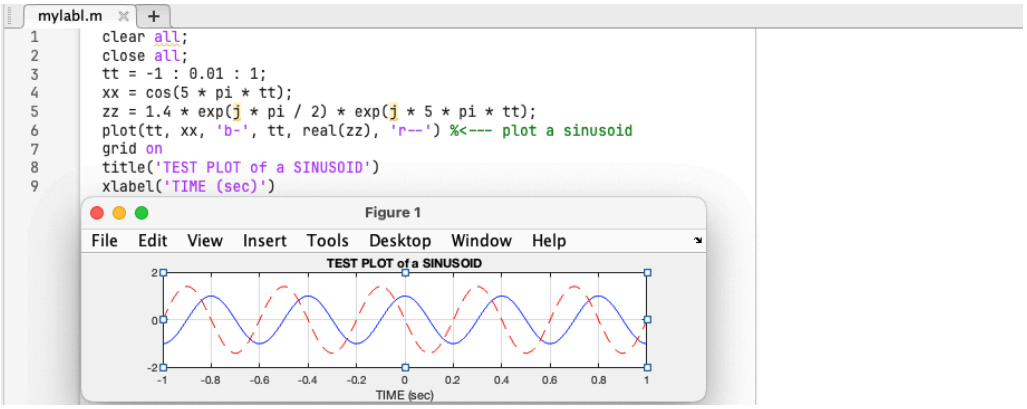
得到结果如下：

其中，红色虚线代表 zz 的实部，蓝色实现代表 xx

对于：

解释为什么 real(zz) 的图形是一个正弦曲线。它的相位和幅度是多少？由所绘图形的时移量计算相位。

对于关键命令 `zz = 1.4 * exp(j * pi / 2) * exp(j * 5 * pi * tt);`，写成



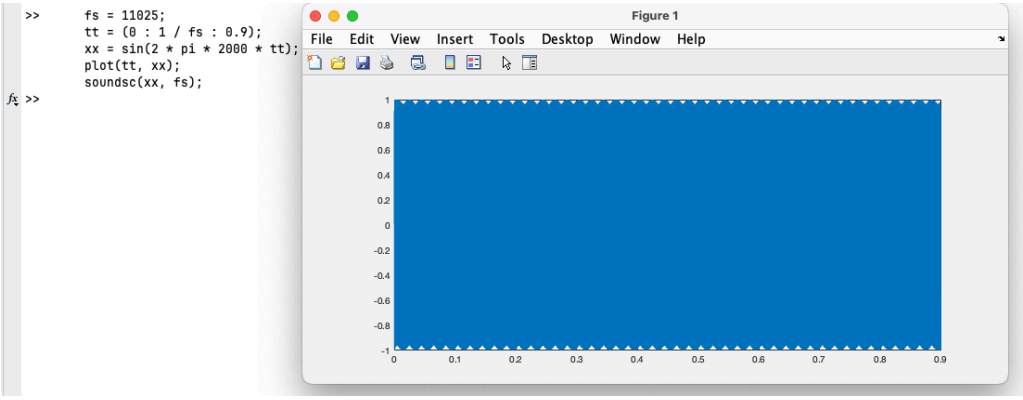
数学表达式就是 $zz = 1.4 * e^{j\frac{\pi}{2}} * e^{j5\pi tt} = j * 1.4e^{j5\pi tt} = -1.4\sin(5\pi tt) + 1.4 * j\cos(5\pi t)$，所以 $real(zz) = -1.4\sin(5\pi tt)$，即为一个正弦函数，其相位为 π，幅度为 1.4。

3.2.3 Matlab 与声音相关的函数命令

要生成的正弦信号的频率为 2000Hz，时长为 0.9 秒，采样率 fs 为 11025 个样本 / 秒，则可以使用以下命令：

```
fs = 11025;
tt = (0 : 1 / fs : 0.9);
xx = sin(2 * pi * 2000 * tt);
plot(tt, xx);
soundsc(xx, fs);
```

命令运行后，可以听到输出的正弦信号声音，绘制得到的 xx 的波形如下：



向量 tt 的长度为 9923。

3.3 用 Matlab 处理正弦信号

3.3.1 处理正弦信号

新建脚本文件，命名为 Processing_sinusoidal_signals.m，要想完成：

在实验报告中要包括本节的图形和实验结果描述。写一个Matlab的m文件来完成步骤（a）到步骤（d）。你们的实验报告也要包括这个m文件的代码。

（a）生成一个时间向量tt，需要覆盖下面的（b）中频率为4000Hz的正弦信号的两个周期。tt的定义方式使用3.2（d）的方式。如果我们使用T表示正弦的周期，定义向量tt的开始时间为－T，结束时间为＋T，那两个周期就会包括tt=0。最后，确保你们正弦波的每个周期有至少25个样本。也就是说，当你们使用冒号操作符定义时间变量时，要使增量足够小，才能保证每个周期可以产生至少25个样本。

（b）生成以下两个4000Hz的正弦序列：

[1] soundsc(xx,fs)函数需用两个参数，,xx 是播放文件的数据的向量，fs 是播放样本的采样率。另外，soundsc(xx,fs)会自动把 xx 的值尺度变化成－1 到 1 之间，然后调用 sound(xx,fs)播放信号。

x1(t) = A1 cos(2π(4000)(t − tm1)) x2(t) = A2 cos(2π(4000)(t − tm2))

其中A1是你的年龄，A2=1.2A1，tm1 = (37.2/M)T ，tm2 = −(41.3/D)T ，这里M和D分别是你生日的月和日，T是周期

绘制以上两个信号在-T≤t≤T的图形。为了输出下面（d）的要求，使用subplot(3,1,1)和subplot(3,1,2)来使图在一个图形窗口内，详见helpwin subplot

一般用法是：

```
subplot(2,1,2);
plot(xx);
```

（c）创建第3个正弦信号为 x3(t) = x1(t) + x2(t). 在Matlab中，这表示把两个正弦向量中的值对应相加。绘制 x3(t) 在-T≤t≤T的图形，使用subplot(3,1,3)绘图。

（d）对每个图都添加一个图名，图名都要包含你的姓名，用title函数

详见 helpwin title，helpwin subplot

根据题目要求和自身情况可知：

$T = \frac{1}{4000}$

设置正弦波每个周期采样有 31 个样本，则 tt 将会有 63 个值（包含 0）；

tt 开始时间为-T，结束时间为 T，则时间向量 tt 的定义为-T 到 T 的 62 个均匀分布的值。

$A_1 = 18$，$A_2 = 1.2 * 18 = 21.6$

$M = 5$，$t_{m1} = (37.2/5)T$；

$D = 23$，$t_{m_2} = -(41.3/23)T$；

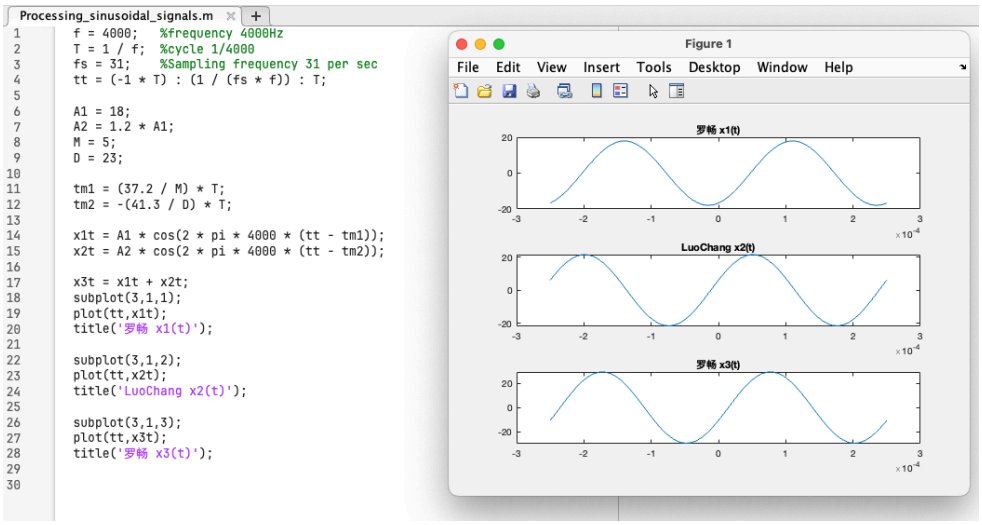
故 $x_1(t) = 18\cos(2\pi \times 4000(t - (37.2/5)T))$，$x_2(t) = 21.6\cos(2\pi \times 4000(t + (41.3/23)T)$，

结合其他实验要求，在脚本文件中输入以下代码：

```
f = 4000;   %frequency 4000Hz
T = 1 / f;  %cycle 1/4000
fs = 31;    %Sampling frequency 31 per sec
tt = (-1 * T) : (1 / (fs * f)) : T;

A1 = 18;
A2 = 1.2 * A1;
M = 5;
D = 23;
```

```
tm1 = (37.2 / M) * T;
tm2 = -(41.3 / D) * T;

x1t = A1 * cos(2 * pi * 4000 * (tt - tm1));
x2t = A2 * cos(2 * pi * 4000 * (tt - tm2));

x3t = x1t + x2t;
subplot(3,1,1);
plot(tt,x1t);
title('罗畅 x1(t)');

subplot(3,1,2);
plot(tt,x2t);
title('LuoChang x2(t)');

subplot(3,1,3);
plot(tt,x3t);
title('罗畅 x3(t)');
```

得到结果如下：

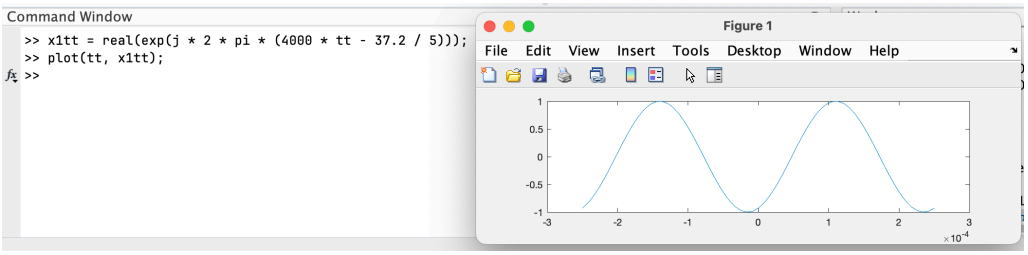


3.3.2 复数幅度

已知 $x_1(t) = 18\cos(2\pi \times 4000(t - (37.2/5)T))$，故可以将 $x_1(t)$ 写为：$Re\{e^{j2\pi(4000t - \frac{37.2}{5})}\}$，在 Matlab 中命令如下：

```
x1tt = real(exp(j * 2 * pi * (4000 * tt - 37.2 / 5)));
plot(tt, x1tt);
```

在 Matlab 中输入以上命令，得到结果如下：



可以看到，$x1tt$ 的图像和 $x1t$ 一模一样，则 $x1tt == x1t$。

3.4 复指数简介

3.4.1 向量化

要想完成：

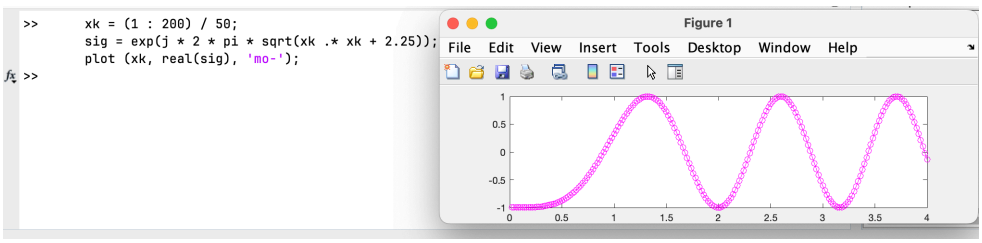
用这种向量化的方法编写 2 - 3 行代码完成以下 Matlab 代码，不使用循环。（注：当 xx 是向量时，xx*xx 和 xx.*xx 是不同的）

```
%--- make a plot of a weird signal
N = 200;
for k = 1 : N
    xk(k) = k / 50;
    rk(k) = sqrt(xk(k) * xk(k) + 2.25);
    sig(k) = exp( j * 2 * pi * rk(k));
end
plot(xk, real(sig), 'mo-')
```

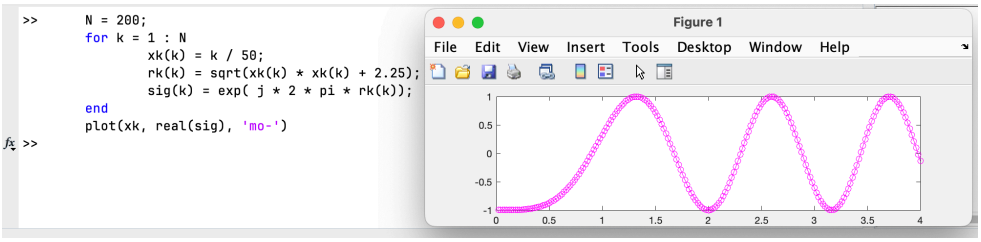
分析可知，xk 为时间向量，rk 为中间变量，sig 是最终所需要的函数，可化简为：

```
xk = (1 : 200) / 50;
sig = exp(j * 2 * pi * sqrt(xk .* xk + 2.25));
plot (xk, real(sig), 'mo-');
```

运行得到结果如下：



运行原代码得到结果如下：



可以看到，图像一样，代码正确。

3.4.2 函数

在 Matlab 中新建脚本文件 testcos.m，输入以下代码：

```
function [xx,tt] = goodcos(ff, dur)
tt = 0 : 1 / (100 * ff) : dur; %-- gives 100 samples per period
xx = cos( 2 * pi * ff * tt);
```

在命令行中输入命令 `testcos(1, 10)`，得到输出为向量 xx 的 1001 个值，而如果输入命令 `goodcos(1, 10)`，则会报错`"Unrecognized function or variable 'goodcos'."`

3.5 复指数

3.5.1 生成正弦信号的 M-文件

要想完成：

> 写一个可以生成单一正弦信号 $x(t) = A\cos(\omega t + \phi)$ 的函数，使用 4 个输入参量：幅度 A，频率 ω，相位 ϕ 和时长 dur。函数应当返回两个输出参量：正弦信号的值 x 和对应的时间 t。确保函数生成的正弦信号在每个周期有 20 个值，函数名为 one_cos()。提示：可借鉴上面的 goodcos() 函数。
> 绘制你们的 one_cos() 函数，参数选为：A = 95，$\omega = 200\pi$ 弧度 / 秒，$\phi = \frac{\pi}{5}$ 弧度，时长为 0.025 秒。推导所绘图形的周期和相位是否正确。如果周期以毫秒为单位是多少？

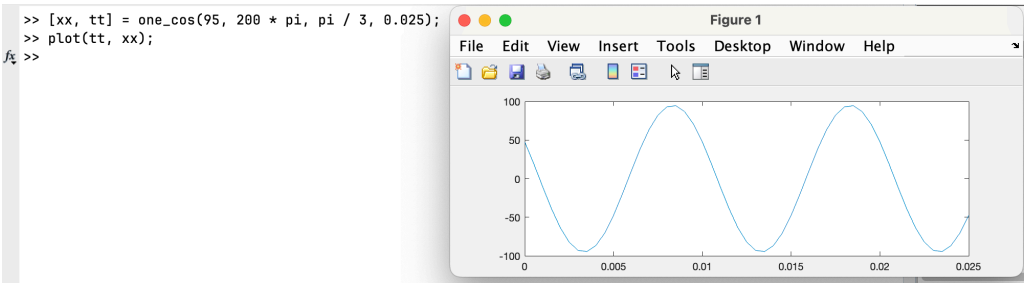
根据要求，在 Matlab 中新建脚本文件 one_cos.m，输入以下代码：

```
function [x, t] = one_cos(A, Ang_freq, phase, dur)
t = 0 : 1 / (20 * (Ang_freq / 2 / pi)) : dur;
x = A * cos(Ang_freq * t + phase);
```

在 Matlab 中输入以下命令：

```
[xx, tt] = one_cos(95, 200 * pi, pi / 3, 0.025);
plot(tt, xx);
```

得到结果如下：



验证图形的正确性：

正弦函数表达式为

$$x(t) = 95\cos(200\pi t + \tfrac{\pi}{3})$$

则其频率 $f = 200\pi/2\pi = 100$，周期 $T = \frac{1}{100}$s 即 10ms，0.025 秒为 2.5 个周期，周期与图像相符；

初相位为 $\phi = \frac{\pi}{3}$，对应幅度为 $95\cos(\frac{\pi}{3}) = 47.5$，相位与图像相符。

3.6 线性调频脉冲 chrip

3.6.1 chirp 信号的 Matlab 合成方法

在 Matlab 中输入以下命令：

```
fsamp = 11025;
dt = 1 / fsamp;
dur = 1.8;
tt = 0 : dt : dur;
psi = 2 * pi * (100 + 200 * tt + 500 * tt .* tt);
xx = real(7.7 * exp(j * psi));
soundsc(xx, fsamp);
```

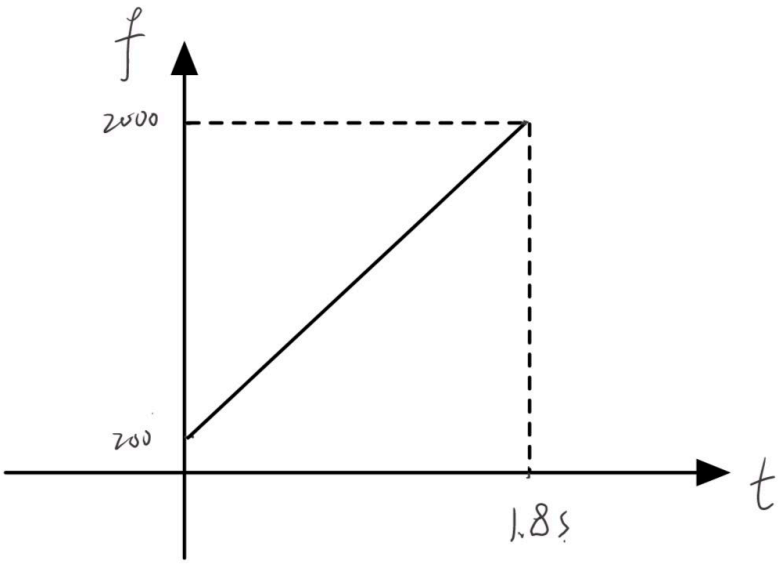
13069 带上耳机可以听到一个音调（频率）从低到高的声音，由代码命令可知：持续时间为 1.8s，时间向量 tt 的长度为 19846；

将以上代码写成数学表达式的形式即为：

$$xx = 7.7\cos(2\pi \times 500tt^2 + 2\pi \times 200tt + 200\pi)$$

故，在以上代码中，T_n 就是 tt，T_s 是 $\frac{1}{fsamp}$，$A = 7.7$，$\mu = 500$，$f_0 = 200$，$\phi = 200\pi$。

tt = 0 时，频率为 200Hz，tt = 1.8 时，频率为 200 + 2 * 500 * 1.8 = 2000Hz，可以绘制图像如图所示：



3.6.2 chirp 的函数

根据要求，在 Matlab 中新建脚本文件 mychirp.m，键入以下内容：

```
function [xx, tt] = mychirp(f1, f2, dur, fsamp)
if(nargin < 4)
    fsamp = 11025;
end
tt = 0 : 1 / (fsamp) : dur;
psi = 2 * pi * (f1 * tt + (f2 - f1) / dur / 2 * tt .* tt);
xx = real(exp(j * psi));
```

在 Matlab 中输入以下命令：

```
[xx, tt] = mychirp(2500, 500, 1.5, 11025);
soundsc(xx,11025);
```

可以听到一个音调逐渐降低的声音，持续 1.5 秒时间。

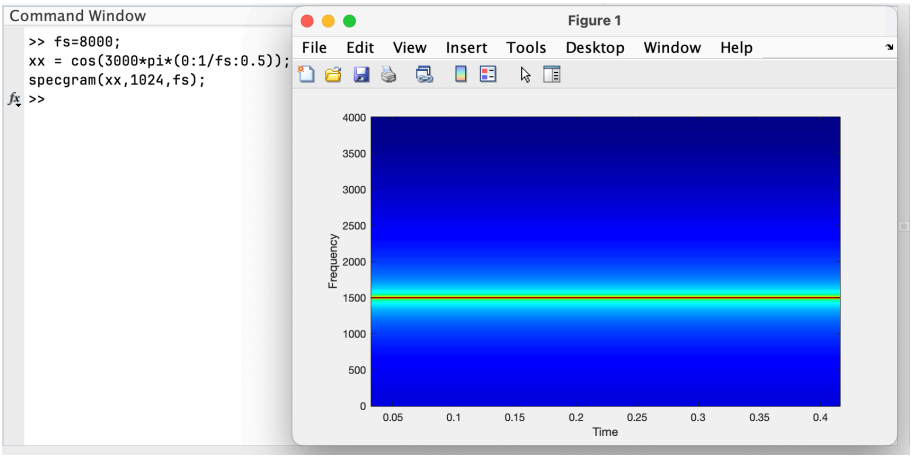
3.6.3 声谱图

在 Matlab 中输入以下命令：

```
fs=8000;
xx = cos(3000 * pi * (0: 1 / fs : 0.5));
```

```
specgram(xx, 1024, fs);
```

可以得到声谱图如下图所示：

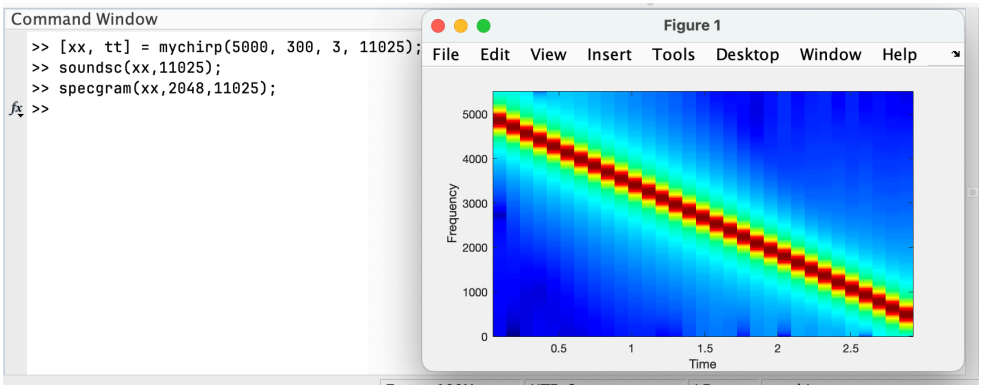


3.6.4 chirp 的声谱图

在 Matlab 中输入以下命令以合成一个新的 chirp 信号：

```
[xx, tt] = mychirp(5000, 300, 3, 11025);
soundsc(xx, 11025);
specgram(xx, 2048, 11025);
```

可以听到一个音调逐渐降低的声音，人耳判断声音频率为线性变化，频率逐渐降低，绘制得到声谱图如下：

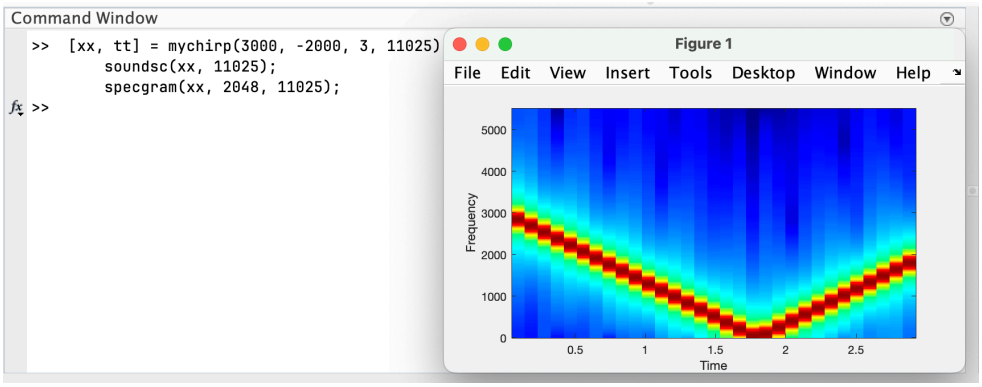


3.6.5 有趣的 chirp

在 Matlab 中输入以下命令以合成一个新的 chirp 信号：

```
[xx, tt] = mychirp(3000, -2000, 3, 11025);
soundsc(xx, 11025);
specgram(xx, 2048, 11025);
```

可以听到一个音调逐渐降低又升高的声音，人耳判断声音频率为线性变化，频率逐渐降低有逐渐升高，绘制得到声谱图如下：



声谱图中表现为频率先降低后升高。根据欧拉公式，可以将正余弦函数表示成复指数函数，而复指数函数代表着旋转，负频率代表顺时针旋转，而正频率代表着逆时针旋，故在频谱中，负频率是正频率的共轭，实际人耳听起来声音的频率还是升高的。

4 实验小结

本次实验内容较多，花了好几天才全部完成，但是也收获满满，本次实验大大提升了我的 Matlab 的使用熟练度，感受到了 Matlab 中矩阵 - 向量语法的便捷与强大，为后续的实验打下了坚实的基础。

2023 年 10 月 30 日　罗畅 于 华中工学院